

IMT Atlantique

Dépt. Mathematical & Electrical Engineering

Technopôle de Brest-Iroise - CS 83818

29238 Brest Cedex 3

Téléphone : +33 (0)2 29 00 13 04

Télécopie : +33 (0)2 29 00 10 12

URL : www.imt-atlantique.fr



Case Study VI

Python and VHDL Implementation of Sobel Edge Detection Filter

Joseph Jabour

Ali AlGhouwayel

Date d'édition : 21 septembre 2026

Version : 1.1



IMT Atlantique

Bretagne-Pays de la Loire

École Mines-Télécom

Sommaire

1. Project Overview	3
2. Theoretical Background	3
3. Python Preprocessing and Reference Generation	3
3.1. Hardware Stimulus Generation	3
3.2. Software Reference (Euclidean Distance)	3
4. System Overview & Architecture	4
4.1. Detailed Block Technical Reference & Student Tasks	4
4.1.1. Shift Register Line Buffer (linebuffer.vhd)	4
4.1.2. 3x3 Window Extractor (window_extractor_3x3.vhd)	4
4.1.3. Sobel Gradients Calculator (sobel_gradients.vhd)	5
4.1.4. Magnitude Calculator (magnitude_calc.vhd)	6
4.1.5. System Controller / FSM (sobel_fsm.vhd)	6
4.1.6. Top-Level System Integration (sobel_system.vhd)	8
4.1.7. Testbench Architecture (tb_sobel_system.vhd) (provided)	9
5. Comparison & Error Analysis Between Software and Hardware Edges	9
5.1. Mathematical Formulation	9
5.2. Post-Processing Guideline	9

Liste des figures

1.	Line Buffer of 100 Stages.	4
2.	3×3 Window Extractor	5
3.	Sobel Gradients Unit	6
4.	Magnitude Calculator Unit	6
5.	Sobel Finite-State Machine	8
6.	Top-Level Sobel Unit	9

Liste des tableaux

1. Project Overview

The objective of this project is to implement a Sobel edge detection filter in two different environments : Python (Software) and VHDL (Hardware).

- In Python, you will prioritize precision by using the true Euclidean distance for magnitude.
- In VHDL, you will prioritize hardware efficiency by implementing the Manhattan distance approximation.
- Finally, you will perform a comparative analysis to quantify the mathematical “error” introduced by the hardware approximation.

2. Theoretical Background

The Sobel operator estimates the image gradient using two 3×3 convolution kernels K_x and K_y :

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Horizontal Kernel Vertical Kernel

Magnitude Metrics

Assume a grayscale image A , the gradients G_x and G_y are computed using the kernels K_x and K_y by convolution :

$$G_x = K_x * A$$

$$G_y = K_y * A$$

- **Euclidean Distance** : The precise magnitude of the gradient vector.

$$G = \sqrt{G_x^2 + G_y^2}$$

- **Manhattan Distance** : A hardware-friendly approximation that avoids expensive square roots and multipliers.

$$G \approx G_x + G_y$$

3. Python Preprocessing and Reference Generation

Given the raw image file (`imagesrcfile.png` or `.jpg`) :

3.1. Hardware Stimulus Generation

Write a Python script to convert the image into the raw binary format required by the VHDL testbench.

- Load the image, resize it to 100×100 pixels, and convert it to grayscale.
- Extract the 8-bit integer value (0-255) for each pixel.
- Convert each integer into an 8-bit binary string.
- Save these strings to `imagesrc.txt` in row-wise manner, strictly formatting it as one pixel per line (10,000 lines total).

3.2. Software Reference (Euclidean Distance)

Implement the Sobel filter entirely in software.

- Apply the 3×3 K_x and K_y convolution kernels to the grayscale version of the image.
- Calculate the precise edge magnitude using the Euclidean distance : $G = \sqrt{G_x^2 + G_y^2}$
- Save the resulting edge map as `edges_python.png` and export the raw float values to `python_ref.txt` for your final error analysis.

4. System Overview & Architecture

The Sobel Filter project implements a hardware accelerator for 2D image processing, specifically configured to stream and filter an image of size 100×100 pixels (`LINE_LENGTH = 100`, `DATA_WIDTH = 8`). The processing pipeline operates synchronously on a clock domain (`clk`) with a synchronous reset (`rst`).

The system architecture coordinates multiple specialized hardware modules :

1. **Line Buffers (`linebuffer.vhd`)** : Delays incoming pixel streams by complete rows to provide simultaneous vertical access to historical pixel rows.
2. **3x3 Window Extractor (`window_extractor_3x3.vhd`)** : Extracts a spatial 3×3 pixel matrix from vertically aligned rows.
3. **Sobel Gradients (`sobel_gradients.vhd`)** : Computes horizontal (G_x) and vertical (G_y) gradient components using parameterized adders/subtractors.
4. **Magnitude Calculator (`magnitude_calc.vhd`)** : Computes the Manhattan distance approximation $|G_x| + |G_y|$.
5. **System Controller / FSM (`sobel_fsm.vhd`)** : Manages image frame boundaries, line buffer filling states, and pipeline write enables.
6. **Adder/Subtractor (`add_sub_n.vhd`)** : Reusable ripple/lookahead-style structural adder/subtractor supporting two's complement arithmetic.

4.1. Detailed Block Technical Reference & Student Tasks

4.1.1. Shift Register Line Buffer (`linebuffer.vhd`)

- **Functionality** : Acts as a delay line of length `LINE_LENGTH` (configured to 100) to store image scanlines.
- **Datapath** : Employs a VHDL array type of `DATA_WIDTH` vectors. When write enable `we = 1`, array slicing shifts elements down the chain, inserting the newest incoming pixel at index 0 and outputting the delayed pixel from the tail.

What Students Should Do :

- Implement the synchronous process with `clk` and `rst`.
- Complete the array slicing assignment for data shifting and register initialization upon reset.

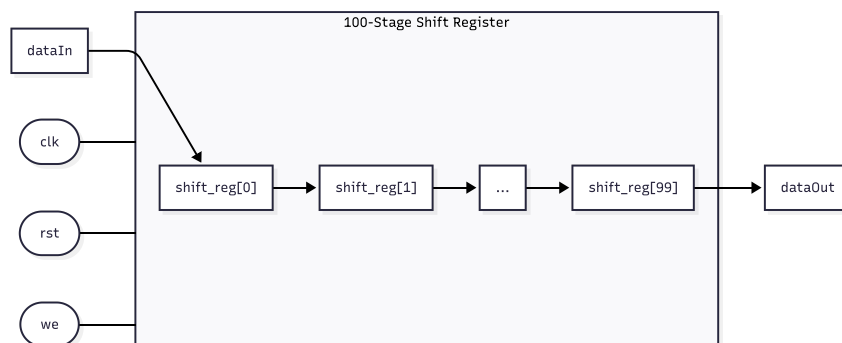


FIGURE 1 – Line Buffer of 100 Stages.

4.1.2. 3x3 Window Extractor (`window_extractor_3x3.vhd`)

- **Functionality** : Takes three vertically aligned pixel inputs from different row layers and extracts a spatial 3×3 window matrix (p_{00} through p_{22}).
- **Datapath** : Implements a bank of registers that shift horizontally upon a valid write enable (`we = 1`).

What Students Should Do :

- Write the synchronous register shifting logic for all three rows in the 3x3 grid.
- Map internal registers to the output ports following the naming convention p_{rowcol} .

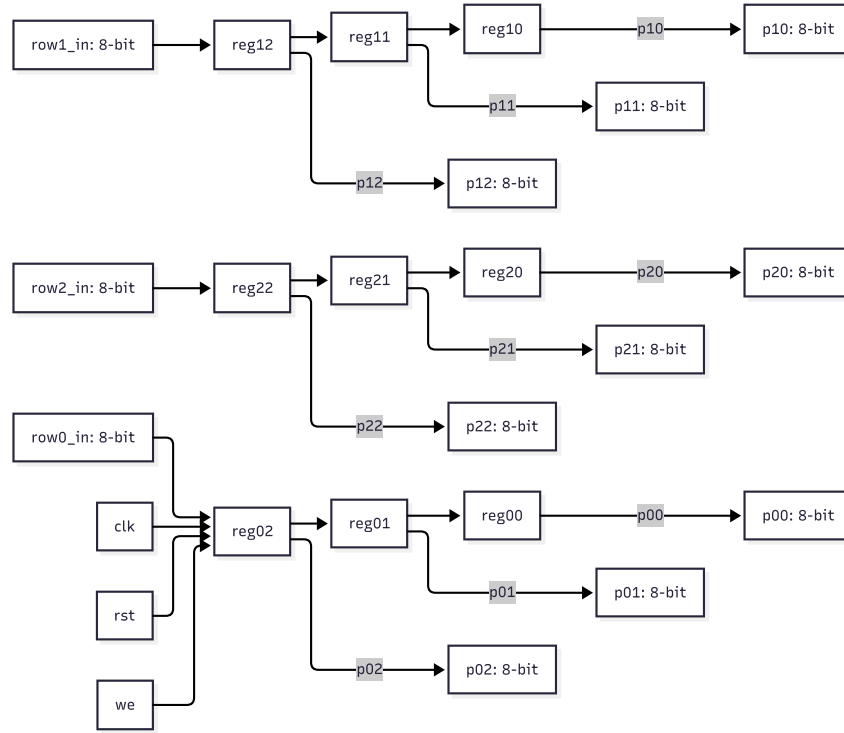


FIGURE 2 – 3×3 Window Extractor

4.1.3. Sobel Gradients Calculator (sobel_gradients.vhd)

- *Functionality* : Computes the horizontal (G_x) and vertical (G_y) gradient components from the 3×3 window pixels using the standard Sobel convolution masks :

$$G_x = (p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20})$$

$$G_y = (p_{00} + 2p_{01} + p_{02}) - (p_{20} + 2p_{21} + p_{22})$$

- *Datapath* : Uses local instantiations of add_sub_n ($N = 11$) with internal padding for sign extension and scaling by 2.

What Students Should Do :

- Instantiate and wire the component add_sub_n for both the G_x and G_y adder trees.
- Properly connect intermediate sum terms to calculate final signed 11-bit outputs gx_out and gy_out.

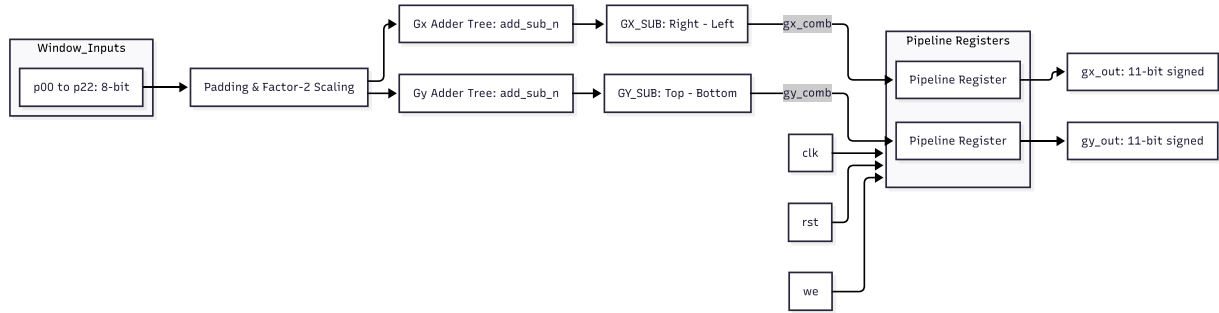


FIGURE 3 – Sobel Gradients Unit

4.1.4. Magnitude Calculator (magnitude_calc.vhd)

- **Functionality** : Computes the approximate edge magnitude using the Manhattan distance metric : $Magnitude = |G_x| + |G_y|$.
- **Datapath** : Computes absolute values of G_x and G_y using the sign-bit subtraction trick ($0 - G_x$ if negative), adds them together, and pipelines the output.

What Students Should Do :

- Complete the structural port mappings for ABS_X_CALC, ABS_Y_CALC, and MAG_ADDER.
- Implement the synchronous output pipeline register process with reset and write-enable handling.

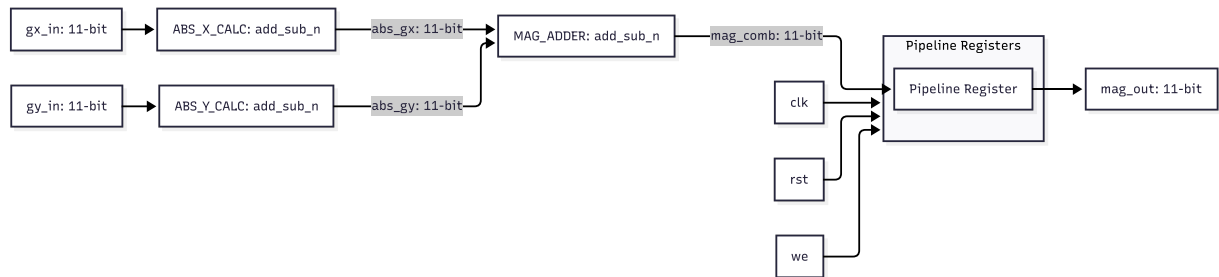


FIGURE 4 – Magnitude Calculator Unit

4.1.5. System Controller / FSM (sobel_fsm.vhd)

1. IDLE :

- **Role** : Acts as the FSM's baseline reset and wait state.
- **Impact on Processing** : Halts all datapath operations. It strictly zeros out col_count and row_count, ensuring any subsequent frame processing begins at coordinate (0,0).
- **Transition Trigger** : Exits to FILL_LB0 only when the start input goes high.

2. FILL_LB0

- **Role** : Processes the first row of incoming pixel data without triggering mathematical operations.
- **Impact on Outputs** : Asserts we_1b (Write Enable Line Buffer) and we_win (Write Enable Window) synchronously with pixel_valid_in. It guarantees no math write-enables or pipeline valid flags are triggered.
- **Impact on State/Counters** : Increments col_count for every valid pixel.
- **Transition Trigger** : When the column reaches IMAGE_WIDTH - 1, it resets col_count to 0, permanently sets row_count to 1, and moves to FILL_LB1.

3. FILL_LB1

- **Role :** Processes the second row of incoming pixel data, continuing to buffer data while keeping math operations dormant.
- **Impact on Outputs :** Identical to `FILL_LB0`. It asserts `we_lb` and `we_win` strictly when `pixel_valid_in` is high. `we_math` remains defaulted to '0'.
- **Impact on State/Counters :** Increments `col_count` with valid inputs.
- **Transition Trigger :** Upon hitting `IMAGE_WIDTH - 1`, it zeros the column counter, sets `row_count` to 2, and pushes the machine into the primary processing phase (`PROCESS_ROW`).

4. PROCESS_ROW

- **Role :** The main processing state. It handles the continuous streaming of all remaining image rows and activates the mathematical datapath.
- **Impact on Outputs :** Asserts `we_lb`, `we_win`, and crucially, `we_math` in sync with `pixel_valid_in`.
- **Impact on Pipeline :** Dynamically drives the control pipeline. It flags `ctrl_pipe(0).valid` to '1' only when `col_count` ≥ 2 (signaling the datapath is loaded). It injects a one-clock-cycle pulse into `ctrl_pipe(0).eof` at the exact end of the frame.
- **Impact on State/Counters :** Manages 2D traversal by wrapping `col_count` at `IMAGE_WIDTH - 1` and incrementing `row_count` accordingly.
- **Transition Trigger :** Once both `col_count` reaches `IMAGE_WIDTH - 1` and `row_count` reaches `IMAGE_HEIGHT - 1` simultaneously, it transitions to `FINISHED`.

5. FINISHED

- **Role :** A terminal lock-state that safely concludes a frame's processing lifecycle.
- **Impact on Processing :** Removes all drive signals from the datapath. Because no specific code is written in this block to override defaults, all `we_*` signals and `ctrl_pipe(0)` inputs fall back to '0'. The synchronization shift register continues to empty out the delayed pipeline values during this phase.
- **Transition Trigger :** Prevents processing of a new frame until the external controller acknowledges completion by dropping `start` to '0', returning the system to `IDLE`.

What Students Should Do :

- Complete the FSM logic based on the provided description and figure 5.

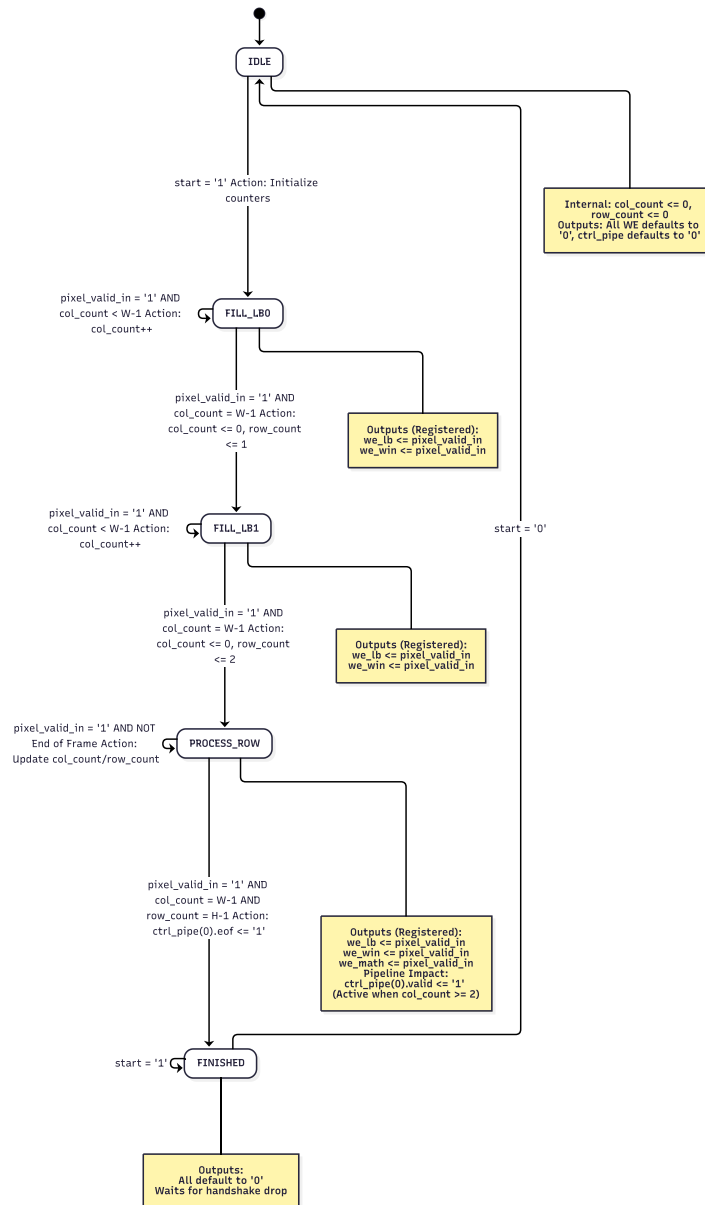


FIGURE 5 – Sobel Finite-State Machine

4.1.6. Top-Level System Integration (sobel_system.vhd)

- **Functionality** : Ties together the Line Buffers, Window Extractor, Gradients module, Magnitude Calculator, and FSM into a cohesive top-level structural design.
- **Datapath** : Routes control signals from sobel_fsm to the respective datapath components and chains intermediate data busses.

What Students Should Do :

- Complete the component instantiations and port mappings connecting all sub-modules.

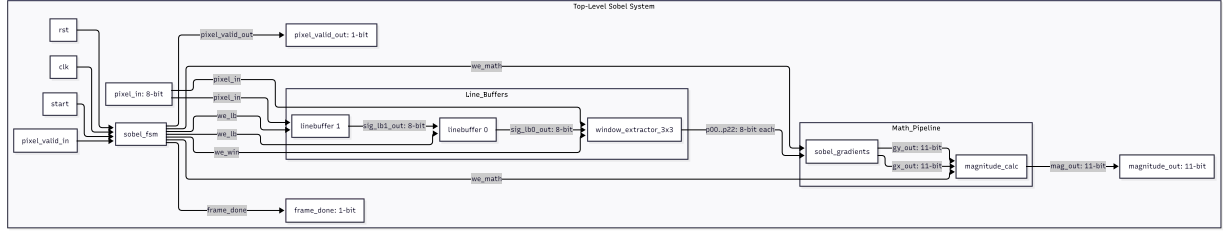


FIGURE 6 – Top-Level Sobel Unit

4.1.7. Testbench Architecture (tb_sobel_system.vhd) (provided)

What the Block Does :

- **Functionality** : Acts as the top-level test harness (DUT wrapper) to verify the correctness of the Sobel Edge Detection hardware system. It handles clock generation, test stimulus file reading, streaming pixel validation, and response logging to an external text file (SobelEdgeVHDL.txt)¹.
- **Datapath & File I/O** :
 1. **Clock Generator Process** : Automatically toggles the internal clk signal at a fixed half-period (CLK_PERIOD / 2).
 2. **Stimulus Driver Process** : Opens input source file imagesrc.txt¹ containing raw pixel test streams, reads line-by-line, drives pixel_in, and pulses pixel_valid_in synchronously with the clock edges.
 3. **Monitor / Scoreboard Process** : Samples magnitude_out whenever pixel_valid_out = '1', converts the 11-bit vector back to a bit vector, writes the results to an output text file, and monitors for the completion flag (frame_done = 1) to terminate the simulation cleanly.

5. Comparison & Error Analysis Between Software and Hardware Edges

To quantitatively evaluate the accuracy of your VHDL hardware accelerator against your software reference model, you must compute the Mean Absolute Error (MAE) across the entire 100×100 image frame.

5.1. Mathematical Formulation

The Mean Absolute Error is defined as the average of the absolute differences between the reference software edge matrix (G_{python}) and the VHDL hardware output stream (G_{vhdl}) :

$$MAE = \frac{1}{H \times W} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} |G_{python}(i, j) - G_{vhdl}(i, j)| \quad (1)$$

where $H = 100$ and $W = 100$ represent the image frame dimensions.

5.2. Post-Processing Guideline

1. **Generate Difference Image (Heatmap)** : Compute the absolute error matrix $D(i, j)$ for every pixel coordinate :

$$D(i, j) = |G_{python}(i, j) - G_{vhdl}(i, j)| \quad (2)$$

Plotting $D(i, j)$ as a 2D heatmap visually highlights where hardware quantization or approximation differences (such as Manhattan vs. Euclidean distance metrics) diverge from software.

2. **Accumulate and Average** : Sum all elements of the difference matrix $D(i, j)$ and divide by the total number of pixels ($100 \times 100 = 10,000$) :

$$\text{Total Error} = \sum_{i=0}^{99} \sum_{j=0}^{99} D(i, j), \quad MAE = \frac{\text{Total Error}}{10000} \quad (3)$$

1. Update the path in the testbench based on your file location.

OUR WORLDWIDE PARTNERS UNIVERSITIES - DOUBLE DEGREE AGREEMENTS



3 CAMPUS



IMT Atlantique Bretagne-Pays de la Loire – <http://www.imt-atlantique.fr/>

Campus de Brest

Technopôle Brest-Iroise
CS 83818
29238 Brest Cedex 3
France
T +33 (0)2 29 00 11 11
F +33 (0)2 29 00 10 00

Campus de Nantes

4, rue Alfred Kastler
CS 20722
44307 Nantes Cedex 3
France
T +33 (0)2 51 85 81 00
F +33 (0)2 99 12 70 08

Campus de Rennes

2, rue de la Châtaigneraie
CS 17607
35576 Cesson Sévigné Cedex
France
T +33 (0)2 99 12 70 00
F +33 (0)2 51 85 81 99



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom

[Choisir une licence Creative Commons
<https://creativecommons.org/choose/>
et l'apposer ici à la place de cette phrase en
enlevant les crochets]